ScrollSense – Assignment 2

Storage, Indexing and Query Processing on Your Own Schema

Student: Mithilesh (roll number da24b048)
Course: DA3406 Data Management, IIT Madras (Modules 2–3)

1. Environment, Data and Method

1.1 Environment

Machine	Intel Core i7-1355U (10 cores, 12 threads), 15.7 GB RAM, SSD, Windows 11, AC power
Software	SQLite 3.50.4 through Python 3.14.7 (module `sqlite3`)
Storage settings	page size 4096 bytes, journal mode WAL, default page cache 2000 KiB (500 pages)
Scale	SCALE = 50, loaded in one transaction with `PRAGMA foreign_keys = ON`
Tools	`EXPLAIN QUERY PLAN`, wall-clock timing in Python (`bench.py`, `concurrency_demo.py`)

1.2 Timing method

- One warm-up run (not counted), then three timed runs; the **median** is reported. The cache is warm.
- Each `TIME` line shows the three runs, the median and a hash of the result rows. Equal hashes mean equal answers.
- Absolute times vary by about 25 percent between runs and between sessions. Compare ratios.

1.3 Row counts at SCALE 50 (handout: about 50 times A1)

The table was produced by `python bench.py scale`. It builds a SCALE = 1 database with the same generator and patches and compares the row counts.

table	SCALE 1	SCALE 50	ratio
AgentSession	2,000	100,000	50.0x
AppUser	5,000	250,000	50.0x
AudioTrack	500	25,000	50.0x
Block	2,500	124,999	50.0x
Comment	1,613	78,643	48.8x
Creator	750	37,500	50.0x
CreatorTierPeriod	750	37,500	50.0x
Follow	15,000	750,000	50.0x
Handle	5,000	250,000	50.0x
Hashtag	50	50	1.0x
Impression	308,409	15,420,333	50.0x
InferredInterest	12,605	630,001	50.0x
InterestCategory	12	12	1.0x
InterestSuppression	2,534	126,007	49.7x
JudgeScore	695	35,249	50.7x
Like_	8,326	420,623	50.5x
ModelPricePeriod	3	3	1.0x
ModerationState	5	5	1.0x
Mute	5,000	249,998	50.0x
PromptTemplateVersion	11	18	1.6x
Recommendation	8,409	420,333	50.0x
Retraction	815	41,953	51.5x
Share	2,150	104,190	48.5x
SignalEvent	1,037	52,325	50.5x
ToolCall	7,527	372,748	49.5x
Turn	7,004	350,275	50.0x
TurnUsage	7,004	350,275	50.0x
TypeAffinityTest	0	0	-
UserRating	316	14,076	44.5x
Video	20,000	1,000,000	50.0x
VideoHashtag	20,000	1,000,000	50.0x
VideoModerationEvent	24,800	1,240,003	50.0x
ViewSegment	113,424	5,672,455	50.0x

All large tables are 48 to 52 times larger, except UserRating (44.5 times, a small random sample). Hashtag, InterestCategory, ModelPricePeriod and ModerationState are fixed lookup tables.

2. Changelog of A1 Changes

The base data needs eight A1 patch scripts. Five were missing and were rebuilt (they are **not** the original A1 code, so the random values differ from A1). All changes are listed.

#	Change	Reason	Effect
1	`generate_data.py`: SCALE applied, one transaction, batched inserts (rewritten before this work).	Speed at 50x.	None. Checked: A1 original and new generator give identical tables at SCALE 1 and SCALE 10 (row by row, rowids included).
2	`patch_missing_data` rebuilt (hashtags, interests, blocks, mutes, recommendations).	Tables empty without it.	Same intent, different values.
3	`fix_view_durations` rebuilt.	All segments had zero length.	Different values.
4	`fix_retraction_gaps` rebuilt.	Retraction time equalled like time.	Different values; F4 returns 60 percent of retractions as in A1.
5	`patch_moderation_history` rebuilt.	F8 was empty.	Different values.
6	`backfill_turn_usage` rebuilt (token ranges are my choice).	TurnUsage was empty.	Different values.
7	`fix_user_activity_pattern` rewritten as one set-based UPDATE (hash instead of random).	Hours of run time at 50x.	Same distribution, not the same values.
8	`patch_toolcall_nesting` rewritten (500 times SCALE turns, one pass).	Speed; keep share of nested calls.	22,364 nested calls.
9	`fix_recommendation_watches`: executemany.	Speed.	None in logic.
10	New files: `apply_a1_patches.py`, `_sqlhash.py`, `verify_db.py`, `bench.py`, `concurrency_demo.py`.	Run and check.	None on data.
11	Patch order as in the A1 README. Known A1 quirk kept: activity patch moves `Impression.ts` only.	Same as A1.	Some date mixes are not exact.
12	`queries.sql`: A1 text unchanged, A2 section appended. `transactions.sql`: (a) T1 used `SELECT 1/0`, which returns NULL in SQLite and does not fail; replaced by a duplicate-key insert with OR ROLLBACK. (b) ROLLBACK added after the open T2 so the file runs as one script. `indexes.sql`: IF NOT EXISTS.	The files must run in DBeaver.	T1 now changes nothing, as its comment says. T3 still changes the handles of user 20: run on a copy.
13	Schema: none. `schema.sql`, `views.sql`, `bonus.sql` not edited.	–	None.
14	Backups of the database (not committed).	Restore point.	None.

Foreign key check after all patches: 0 violations. Queries F2, F4, F7, F11 and F12 return non-trivial results (F2 37,500 rows, F4 25,164, F7 37,926, F11 7, F12 128,805).

3. Task 1: Access Paths and Cost Arithmetic (20 marks)

Question

Pick three queries from `queries.sql`; at least one joins two or more high-volume tables.

- **1.1** Paste each plan and classify the access path of every table: full scan, rowid/PK lookup, or existing index. (8)
- **1.2** For one full-table-scan step: row count, assumed block size, rows per block, blocks transferred, arithmetic shown. (7)
- **1.3** One sentence per query: why this path and not the next best. (5)

Code

Database: SCALE = 50, no secondary index. Queries (unchanged from `queries.sql`): **F1** (Video, AudioTrack), **F11** (ToolCall, Turn, recursive), **F12** (Recommendation, Turn, Video, AgentSession, Impression, ViewSegment). F12 joins the high-volume tables Impression, ViewSegment, Video, Turn and AgentSession.

```
EXPLAIN QUERY PLAN <query>;          -- in DBeaver, or:  python bench.py task1
```

Results

1.1 Plans (raw output; the numbers in brackets are SQLite's node and parent id):

```
EXPLAIN QUERY PLAN F1
  [9<-0] SCAN v
  [14<-0] SCALAR SUBQUERY 1
  [19<-14] SEARCH Video
  [31<-0] SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
  [34<-0] USE TEMP B-TREE FOR GROUP BY
  [83<-0] USE TEMP B-TREE FOR ORDER BY

EXPLAIN QUERY PLAN F11
  [2<-0] CO-ROUTINE call_tree
```

```
  [5<-2] SETUP
  [7<-5] SCAN ToolCall
  [12<-5] LIST SUBQUERY 1
  [15<-12] SEARCH Turn USING COVERING INDEX sqlite_autoindex_Turn_1 (session_id=?)
  [23<-12] CREATE BLOOM FILTER
  [54<-2] RECURSIVE STEP
  [56<-54] SCAN ct
  [59<-54] BLOOM FILTER ON tc (parent_call_id=?)
  [71<-54] SEARCH tc USING AUTOMATIC COVERING INDEX (parent_call_id=?)
  [92<-0] SCAN call_tree
  [103<-0] USE TEMP B-TREE FOR ORDER BY

EXPLAIN QUERY PLAN F12
  [12<-0] SCAN r USING INDEX sqlite_autoindex_Recommendation_1
  [15<-0] SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
  [18<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [24<-0] SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  [29<-0] BLOOM FILTER ON i (video_id=? AND user_id=?)
  [39<-0] SEARCH i USING AUTOMATIC COVERING INDEX (video_id=? AND user_id=?)
  [48<-0] BLOOM FILTER ON vs (impression_id=?)
  [59<-0] SEARCH vs USING AUTOMATIC COVERING INDEX (impression_id=?)
```

Query	Table	Access path	Plan line
F1	Video (main)	full scan	`SCAN v`
F1	Video (MAX subquery)	full scan (bytecode: Rewind, AggStep, Next)	`SEARCH Video`
F1	AudioTrack	rowid / PK lookup	`SEARCH at USING INTEGER PRIMARY KEY`
F11	ToolCall (start)	full scan	`SCAN ToolCall`
F11	Turn	existing index (UNIQUE session_id, turn_index), covering	`SEARCH Turn USING COVERING INDEX sqlite_autoindex_Turn_1`
F11	ToolCall (recursion)	temporary automatic index on parent_call_id	`SEARCH tc USING AUTOMATIC COVERING INDEX`
F12	Recommendation	existing index (PK), scanned fully	`SCAN r USING INDEX sqlite_autoindex_Recommendation_1`
F12	Turn, Video, AgentSession	rowid / PK lookup	`SEARCH t / v / s USING INTEGER PRIMARY KEY`
F12	Impression, ViewSegment	temporary automatic index built at run time	`SEARCH i / vs USING AUTOMATIC COVERING INDEX`

Times:

```
TIME F1 as written: rows=10  runs(s)=0.335, 0.344, 0.269  median=0.335s  result_hash=d4bf73d592

TIME F11 as written: rows=7  runs(s)=0.224, 0.224, 0.226  median=0.224s  result_hash=0b0d718dc8
```

1.2 Hand calculation: the `SCAN ToolCall` step of F11.

1. Block size $P = 4096$ bytes (the SQLite page; assumed block).
2. Row count $n = 372{,}748$ (`SELECT COUNT(*) FROM ToolCall`).
3. Average row (cell) size s = record payload + length varint + rowid varint + 2-byte pointer. The payload is a header (1 byte + one serial-type byte per column) plus the column data; the key `call_id` is the rowid and takes no space.
4. Rows per page = $\lfloor (P - 8)/s \rfloor$ (a leaf page has an 8-byte header).
5. Pages = $\lceil n / \text{rows per page} \rceil$.

```
table                = ToolCall
block size (page)    = 4096 bytes   (PRAGMA page_size; I assume block = page)
row count (n)        = 372,748
avg record payload   = 126.23 bytes (largest row 128 bytes, so no row needs an overflow page: the limit
       is about 4061)
avg cell size        = 132.94 bytes = payload 126.23 + payload-length varint 1.76 + rowid varint 2.96 +
       cell pointer 2
usable bytes / page  = 4096 - 8 = 4088
rows per page        = floor(4088 / 132.94) = 30
blocks to read       = ceil(372,748 / 30) = 12,425 pages = 50.9 MB
CHECK (real)         : a fresh database holding the same 372,748 rows in rowid order uses 12,300 pages
estimate minus real  : +125 pages (+1.02 percent)

SCAN PROBE  SELECT SUM(latency_ms) FROM ToolCall (reads every page of the table): runs(s)=0.040, 0.041,
       0.042  median=0.041s
           = 301,035 pages per second, 0.11 microseconds per row (warm cache)
```

Cost: reading the whole table is 12,425 block transfers by the estimate and 12,300 in a real copy (difference 1.02 percent).

Answer

1.3 One sentence per query.

- **F1:** no index exists on `uploaded_at` or `audio_track_id`, so Video is scanned and each AudioTrack is found by primary key; the alternative (start from AudioTrack, look up videos) needs an index that does not exist.
- **F11:** ToolCall has no index on `turn_id` or `parent_call_id`, so the start rows come from a scan; the turns of one session come from the existing UNIQUE index on Turn; one temporary index on `parent_call_id` is used at every recursion level, because re-scanning 372,748 rows per level would cost far more.
- **F12:** it starts from the small Recommendation table through its primary key and uses primary keys for Turn, Video and AgentSession; Impression and ViewSegment have no usable index, so SQLite builds temporary ones, since scanning 15.4 million impressions per recommendation would be hopeless.

Conclusion. Without secondary indexes the queries use full scans, primary key lookups and temporary indexes. The page arithmetic matches the real page count within about 1 percent.

4. Task 2: Composite Index Design (25 marks)

Question

Choose three Deliverable-F queries that use at least two columns of the same table.

- **2.1** One `CREATE INDEX` each; justify the column order by the leftmost-prefix rule: which clause each column serves and what another order leaves unused. (12)
- **2.2** Plan and wall-clock time before and after each index; an index that is ignored or does not help is valid if diagnosed. (9)
- **2.3** One index considered and rejected before building, with the reason. (4)

Code

Rule used: an index can be searched only through its first columns, in order. Order: equality columns, then one range column, then sort/group columns, then columns that only make the index covering.

```
CREATE INDEX idx_video_audio_uploaded  ON Video(audio_track_id, uploaded_at);                    -- F1
CREATE INDEX idx_like_retracted_cover  ON Like_(retracted_at, liked_at, user_id, video_id);       -- F4
CREATE INDEX idx_impression_video_user ON Impression(video_id, user_id);                          -- F12
```

Commands: `python bench.py task2` (full run), `python bench.py task2more` (write cost, MAX probe, rejected index). In DBeaver: run `indexes.sql` (Alt+X).

Results

2.1 Column order, clause by clause.

Query	Index	Clause served by each column	Other order
F1	`Video(audio_track_id, uploaded_at)`	`audio_track_id`: JOIN equality (first); `uploaded_at`: WHERE range (second). Covering.	`(uploaded_at, audio_track_id)`: the equality cannot seek after a range; reads 26.7 percent of the index.
F4	`Like_(retracted_at, liked_at, user_id, video_id)`	`retracted_at`: WHERE `IS NOT NULL` (the only seekable test, 10 percent of rows); `liked_at`: used in the 60-second calculation; `user_id`, `video_id`: SELECT list (covering).	`(liked_at, ...)`: no condition on `liked_at` alone, so nothing can seek.
F12	`Impression(video_id, user_id)`	Two JOIN equalities. `video_id` first: more selective (1,000,000 distinct values against 250,000) and its prefix also serves other joins.	Either order works for F12; the prefix `video_id` would be lost for other joins.

2.2 Plans and times (raw output; median of three runs).

F1 before / after:

```
EXPLAIN QUERY PLAN F1
  [9<-0] SCAN v
  [14<-0] SCALAR SUBQUERY 1
```

```
  [19<-14] SEARCH Video
  [31<-0] SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
  [34<-0] USE TEMP B-TREE FOR GROUP BY
  [83<-0] USE TEMP B-TREE FOR ORDER BY

TIME F1 before: rows=10  runs(s)=0.288, 0.289, 0.300  median=0.289s  result_hash=d4bf73d592

EXPLAIN QUERY PLAN F1
  [9<-0] SCAN at
  [11<-0] SEARCH v USING COVERING INDEX idx_video_audio_uploaded (audio_track_id=? AND uploaded_at>?)
  [15<-0] SCALAR SUBQUERY 1
  [20<-15] SEARCH Video USING COVERING INDEX idx_video_audio_uploaded
  [74<-0] USE TEMP B-TREE FOR count(DISTINCT)
  [77<-0] USE TEMP B-TREE FOR ORDER BY

TIME F1 after: rows=10  runs(s)=0.116, 0.109, 0.115  median=0.115s  result_hash=d4bf73d592
```

F1 control, other order (uploaded_ at first):

```
EXPLAIN QUERY PLAN F1 with other order
  [9<-0] SEARCH v USING COVERING INDEX idx_wrong_order (uploaded_at>?)
  [12<-0] SCALAR SUBQUERY 1
  [17<-12] SEARCH Video USING COVERING INDEX idx_wrong_order
  [33<-0] SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
  [36<-0] USE TEMP B-TREE FOR GROUP BY
  [83<-0] USE TEMP B-TREE FOR count(DISTINCT)
  [86<-0] USE TEMP B-TREE FOR ORDER BY

TIME F1 other order: rows=10  runs(s)=0.132, 0.134, 0.136  median=0.134s  result_hash=d4bf73d592
```

F4 before / after:

```
EXPLAIN QUERY PLAN F4
  [2<-0] SCAN l

TIME F4 before: rows=25,164  runs(s)=0.051, 0.050, 0.052  median=0.051s  result_hash=24c86ed4e0

EXPLAIN QUERY PLAN F4
  [2<-0] SEARCH l USING COVERING INDEX idx_like_retracted_cover (retracted_at>?)

TIME F4 after: rows=25,164  runs(s)=0.030, 0.029, 0.030  median=0.030s  result_hash=24c86ed4e0
```

F4 control, other order (liked_ at first): index ignored.

```
EXPLAIN QUERY PLAN F4 with other order
  [2<-0] SCAN l

TIME F4 other order: rows=25,164  runs(s)=0.049, 0.050, 0.049  median=0.049s  result_hash=24c86ed4e0
```

F12 before / after:

```
EXPLAIN QUERY PLAN F12
  [12<-0] SCAN r USING INDEX sqlite_autoindex_Recommendation_1
  [15<-0] SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
  [18<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [24<-0] SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  [29<-0] BLOOM FILTER ON i (video_id=? AND user_id=?)
  [39<-0] SEARCH i USING AUTOMATIC COVERING INDEX (video_id=? AND user_id=?)
  [48<-0] BLOOM FILTER ON vs (impression_id=?)
  [59<-0] SEARCH vs USING AUTOMATIC COVERING INDEX (impression_id=?)

TIME F12 before: rows=128,805  runs(s)=236.325, 215.335, 220.275  median=220.275s
       result_hash=8dfe7302fc

EXPLAIN QUERY PLAN F12
  [12<-0] SCAN r USING INDEX sqlite_autoindex_Recommendation_1
  [15<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [21<-0] SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
  [24<-0] SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  [27<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=? AND user_id=?)
  [41<-0] BLOOM FILTER ON vs (impression_id=?)
  [52<-0] SEARCH vs USING AUTOMATIC COVERING INDEX (impression_id=?)

TIME F12 after: rows=128,805  runs(s)=9.037, 8.970, 8.930  median=8.970s  result_hash=8dfe7302fc
```

F12 control, with an extra index on ViewSegment(impression_ id):

```
EXPLAIN QUERY PLAN F12 with both indexes
  [13<-0] SCAN r USING INDEX sqlite_autoindex_Recommendation_1
  [16<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [22<-0] SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
  [25<-0] SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  [28<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=? AND user_id=?)
  [40<-0] SEARCH vs USING INDEX idx_extra_seg_impression (impression_id=?)

TIME F12 with both indexes: rows=128,805  runs(s)=6.598, 6.514, 6.359  median=6.514s
       result_hash=8dfe7302fc
```

Summary (median seconds; sizes in 4096-byte pages from the build lines of the run):

Query	Before (s)	After (s)	Faster by	Index size
F1	0.289	0.115	2.5x	7,724 pages (31.6 MB)
F1, other order	–	0.134	2.2x	7,708 pages
F4	0.051	0.030	1.7x	4,112 pages (16.8 MB)
F4, other order	–	0.049 (ignored)	1.0x	4,116 pages
F12	220.3	8.970	24.6x	61,623 pages (252.4 MB)

Extra write cost (scratch copy of the first rows; the real database is not touched): index maintenance on bulk and on single-row inserts.

```
WRITE COST F1 index: table Video, 300,000 rows preloaded, then 50,000 rows in one transaction, then
       1,000 single-row transactions (median of 3)
   bulk insert   :      49.1 ms without index,     651.5 ms with index  ->   0.98 and  13.03
       microseconds per row, +1228 percent
   single inserts:    1266.7 ms without index,    1294.7 ms with index  ->  1.267 and  1.295 ms per row,
       +2 percent
   pages used after the preload: 4,850 without index, 7,384 with index -> the index adds 2,534 pages =
       10.4 MB for 300,000 rows

WRITE COST F4 index: table Like_, 300,000 rows preloaded, then 50,000 rows in one transaction, then
       1,000 single-row transactions (median of 3)
   bulk insert   :      42.6 ms without index,     780.4 ms with index  ->   0.85 and  15.61
       microseconds per row, +1730 percent
   single inserts:    1247.6 ms without index,    1294.0 ms with index  ->  1.248 and  1.294 ms per row,
       +4 percent
   pages used after the preload: 2,943 without index, 6,208 with index -> the index adds 3,265 pages =
       13.4 MB for 300,000 rows

WRITE COST F12 index: table Impression, 300,000 rows preloaded, then 50,000 rows in one transaction,
       then 1,000 single-row transactions (median of 3)
   bulk insert   :      45.1 ms without index,     445.6 ms with index  ->   0.90 and   8.91
       microseconds per row, +889 percent
   single inserts:    1282.8 ms without index,    1291.7 ms with index  ->  1.283 and  1.292 ms per row,
       +1 percent
   pages used after the preload: 3,670 without index, 4,944 with index -> the index adds 1,274 pages =
       5.2 MB for 300,000 rows
```

MAX subquery of F1 for each column order:

```
TIME MAX subquery, no index: rows=1  runs(s)=0.095, 0.094, 0.104  median=0.095s  result_hash=ae9446ef96

TIME MAX subquery, audio_track_id first: rows=1  runs(s)=0.068, 0.068, 0.065  median=0.068s
       result_hash=ae9446ef96

TIME MAX subquery, uploaded_at first: rows=1  runs(s)=0.000, 0.000, 0.000  median=0.000s
       result_hash=ae9446ef96
```

2.3 Rejected index (not created): `Turn(occurred_at, model_id, priced_at, session_id, template_id, template_version)` for F7.

```
Turn rows: 350,275; rows inside the F7 window: 350,275; selectivity of the range = 100.0 percent

TIME F7 no index: rows=37,926  runs(s)=0.350, 0.342, 0.355  median=0.350s  result_hash=d0c3bb24e9

WRITE COST rejected Turn index: table Turn, 250,000 rows preloaded, then 50,000 rows in one transaction,
       then 1,000 single-row transactions (median of 3)
   bulk insert   :     130.1 ms without index,     804.3 ms with index  ->   2.60 and  16.09
       microseconds per row, +518 percent
   single inserts:    1249.9 ms without index,    1276.5 ms with index  ->  1.250 and  1.277 ms per row,
       +2 percent
   pages used after the preload: 7,778 without index, 11,062 with index -> the index adds 3,284 pages =
       13.5 MB for 250,000 rows
```

Answer

- **F12** gains most: without a permanent index SQLite builds a temporary index on 15.4 million impressions in every run. The permanent index is built once. A control index on ViewSegment removes the second temporary index; the rest of the time is not explained.
- **F1:** the order that follows the rule (equality, then range) is faster than the other order. The other order answers `MAX(uploaded_at)` instantly (see MAX probe), so it is better for queries that filter on `uploaded_at` alone.
- **F4:** gain is small because the query is already fast. The control index (`liked_at` first) is **ignored** by the planner: no condition on its first column, and it is as large as the table (4,116 against 4,126 pages), so scanning it saves nothing.
- **Write cost:** bulk inserts are about 6 to 18 times slower with an index; single-row commits show no measurable difference (about 1.3 ms each, dominated by the commit).
- **Rejected index:** the F7 date window contains every Turn row (selectivity 100 percent), so the index cannot seek; F7 runs in a fraction of a second, so there is little to gain; the index costs space and slows bulk inserts.

5. Task 3: Join Algorithms (20 marks)

Question

- **3.1** Two queries that join two large tables (no lookup tables); state the join strategy from the plan. (8)
- **3.2** For one of them: input cardinalities and cost of nested-loop, sort-merge and hash join by formulas; the cheapest; does the engine agree. (7)
- **3.3** A three-table join: the join order I believe is cheapest from estimated intermediate sizes; if the engine disagrees, who is right and why. (5)

Code

```
J1: SELECT COUNT(*) FROM ViewSegment vs JOIN Impression i ON i.impression_id = vs.impression_id;
J2: SELECT COUNT(*) FROM Like_ l JOIN Impression i ON i.user_id = l.user_id AND i.video_id = l.video_id;
T3: SELECT COUNT(*) FROM Video v JOIN Impression i ON i.video_id = v.video_id
    JOIN ViewSegment vs ON vs.impression_id = i.impression_id WHERE v.owner_id = 12;
```

Commands: `python bench.py task3`, `python bench.py task3stats`. **Note:** SQLite has only nested-loop joins (through the primary key, an existing index, or a temporary *automatic* index). It has **no hash join and no sort-merge join**; it also uses sorters (`USE TEMP B-TREE`) for GROUP BY, DISTINCT and ORDER BY. So 3.2 compares formulas.

Results

3.1 Plans and strategy.

```
EXPLAIN QUERY PLAN J1  ViewSegment JOIN Impression on impression_id
  [4<-0] SCAN vs
  [6<-0] SEARCH i USING INTEGER PRIMARY KEY (rowid=?)

EXPLAIN QUERY PLAN J2  Like_ JOIN Impression on (user_id, video_id)
  [4<-0] SCAN l
  [8<-0] BLOOM FILTER ON i (user_id=? AND video_id=?)
  [18<-0] SEARCH i USING AUTOMATIC COVERING INDEX (user_id=? AND video_id=?)


TIME J1 execution: rows=1  runs(s)=1.466, 1.450, 1.453  median=1.453s  result_hash=a606d1524e

EXPLAIN QUERY PLAN J2  Like_ JOIN Impression on (user_id, video_id)
  [4<-0] SCAN l
  [8<-0] BLOOM FILTER ON i (user_id=? AND video_id=?)
  [18<-0] SEARCH i USING AUTOMATIC COVERING INDEX (user_id=? AND video_id=?)

TIME J2 execution, no index (SQLite builds a temporary index on 15.4M Impression rows in every run):
        rows=1  runs(s)=199.307, 200.841, 197.228  median=199.307s  result_hash=6888b63e74
```

J2 with the permanent index `Impression(video_id, user_id)`:

```
EXPLAIN QUERY PLAN J2 with index
  [4<-0] SCAN l
  [6<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=? AND user_id=?)

TIME J2 execution, with the permanent index: rows=1  runs(s)=2.753, 2.656, 2.646  median=2.656s
        result_hash=6888b63e74
```

- **J1:** nested loop; outer = full scan of ViewSegment, inner = primary key lookup in Impression.
- **J2:** nested loop; outer = full scan of Like_, inner = temporary automatic covering index over all Impression rows (a permanent index replaces it).

3.2 Cost of J1 (block transfers). Symbols: n_r, b_r = rows, pages of ViewSegment; n_s, b_s = rows, pages of Impression; M = memory pages (default cache, 500). Pages were measured by copying each table to a fresh database.

- Block nested loop: $b_r + \lceil b_r/(M-2) \rceil \, b_s$. Tuple nested loop: $b_r + n_r b_s$.
- Index nested loop: $b_r + n_r \, c$, with c = lookup cost (tree height + 1).
- Sort-merge: $2b(1 + \lceil \log_{M-1}(b/M) \rceil)$ for each unsorted input, plus $b_r + b_s$.
- Hash join: $b_r + b_s$ if the build side fits in memory, otherwise $3(b_r + b_s)$.

```
r = ViewSegment: n_r = 5,672,455 rows, b_r = 76,984 pages (measured by copying into a fresh database)
s = Impression : n_s = 15,420,333 rows, b_s = 192,597 pages (measured the same way)
buffer M = default cache 2000 KiB / 4096 B = 500 pages
b-tree of s: interior fanout about 408, interior levels h = ceil(log_408(192,597)) = 3, one lookup = h +
        1 = 4 page reads (worst case)

nested loop, block (outer r):  b_r + ceil(b_r / (M-2)) * b_s = 76,984 + 155 * 192,597 = 29,929,519
nested loop, index (PK of s):  b_r + n_r * (h+1)             = 76,984 + 5,672,455 * 4 = 22,766,804
        (worst case, nothing cached)
      same, interior pages cached (they are only about 473 pages): b_r + n_r * 1 = 5,749,439
sort-merge: sort r = 2*b_r*(ceil(log_499(ceil(b_r/M))) + 1) = 2*76,984*(1+1) = 307,936; s already in key
```

```
       order;
           merge = b_r + b_s ; total = 307,936 + 76,984 + 192,597 = 577,517
hash join (r is build side, does not fit in M): 3*(b_r + b_s) = 3*(76,984 + 192,597) = 808,743
   sort-merge                                    577,517
   hash join                                     808,743
   index nested loop (interior cached)         5,749,439
   index nested loop (worst)                  22,766,804
   block nested loop                          29,929,519
cheapest by formula: sort-merge

nested loop WITHOUT any index, tuple at a time (one full scan of s for every row of r): b_r + n_r * b_s
       = 76,984 + 5,672,455 * 192,597 = 1,092,497,892,619  (shown for scale only)

Effect of the memory size M on the cost (block transfers). Sort: 0 extra transfers if r fits in memory,
       otherwise 2*b_r*(1 + ceil(log_(M-1)(runs))).
Hash join: b_r + b_s if r (the build side) fits in M - 2 pages, otherwise 3*(b_r + b_s).
 M (pages)   M (MB)        block NL    sort-merge     hash join
        50      0.2     309,002,572       731,485       808,743
       500      2.0      29,929,519       577,517       808,743
     5,000     20.5       3,158,536       577,517       808,743
    50,000    204.8         462,178       577,517       808,743
   100,000    409.6         269,581       269,581       269,581


locality check: in segment_id order, impression_id goes DOWN 0 times out of 5,672,455 rows.
So the PK lookups of the engine's nested loop walk the Impression b-tree almost in order (like a merge),
       not at random.
```

3.3 Three tables (creator 12).

```
STATISTICS from the data (these are the selectivities, measured with COUNT(DISTINCT ...)):
  Video: 1,000,000 rows, 107,745 distinct owner_id  -> selectivity of owner_id = ? is 1/107,745; videos
       per owner: min 1, average 9.28, max 46
  Impression: 15,420,333 rows, 1,000,000 distinct video_id -> 15.42 impressions per video
  ViewSegment: 5,672,455 rows, 5,672,455 distinct impression_id -> 0.368 segments per impression (36.8
       percent of impressions have a segment)
  creator 12 really has 30 videos (the uniform estimate is 9.3)
```

Creator-aware estimate (creators own most videos; measured with SQL):

```
Creators: 37,500. Videos owned by creators: 914,911 of 1,000,000 (91.5 percent). Is user 12 a creator?
       yes.
Average videos per creator = 914,911 / 37,500 = 24.40
Impressions per video = 15,420,333 / 1,000,000 = 15.42; segments per impression = 5,672,455 / 15,420,333
       = 0.368
ESTIMATE for a creator, order A : 24.4 videos -> 376 impressions -> 138 segments
ACTUAL for creator 12       : 30 videos -> 452 impressions -> 168 segments
error of the estimate: -19 percent, -17 percent, -18 percent
```

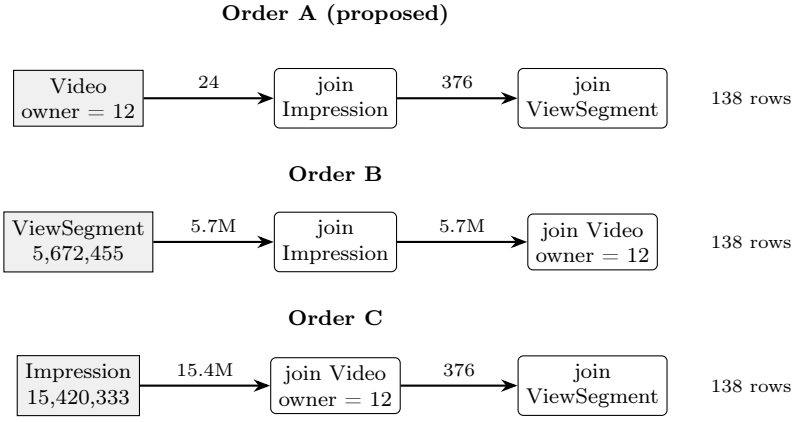


Figure 1. Three join orders with estimated intermediate sizes.

Engine choices, real counts, and forced orders (`CROSS JOIN` keeps the written order):

```
Engine's own choice with NO helper indexes:
EXPLAIN QUERY PLAN T3 no indexes
  [5<-0] SCAN vs
  [7<-0] SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
  [10<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)

TIME T3 engine's order, no helper indexes: rows=1  runs(s)=25.910, 28.284, 67.825  median=28.284s
       result_hash=5f8f164255

Engine's own choice WITH helper indexes (Video(owner_id), Impression(video_id,user_id),
       ViewSegment(impression_id)):
EXPLAIN QUERY PLAN T3 with indexes
  [5<-0] SEARCH v USING COVERING INDEX idx_t3_video_owner (owner_id=?)
  [11<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=?)
  [15<-0] SEARCH vs USING COVERING INDEX idx_t3_seg_impression (impression_id=?)

TIME T3 engine's order: rows=1  runs(s)=0.002, 0.004, 0.008  median=0.004s  result_hash=5f8f164255
```

```
Actual intermediate sizes (real counts):
  videos of the creator   : 30
  impressions of those    : 452
  segments (final count)  : 168


Forced orders (CROSS JOIN makes SQLite keep the written order):
EXPLAIN QUERY PLAN A video,impression,segment
  [5<-0] SEARCH v USING COVERING INDEX idx_t3_video_owner (owner_id=?)
  [11<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=?)
  [15<-0] SEARCH vs USING COVERING INDEX idx_t3_seg_impression (impression_id=?)
TIME order A video,impression,segment: rows=1  runs(s)=0.006, 0.007, 0.006  median=0.006s
       result_hash=5f8f164255
EXPLAIN QUERY PLAN B segment,impression,video
  [5<-0] SCAN vs USING COVERING INDEX idx_t3_seg_impression
  [7<-0] SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
  [10<-0] SEARCH v USING COVERING INDEX idx_t3_video_owner (owner_id=? AND rowid=?)
TIME order B segment,impression,video: rows=1  runs(s)=6.013, 6.414, 6.248  median=6.248s
       result_hash=5f8f164255
EXPLAIN QUERY PLAN C impression,video,segment
  [5<-0] SCAN i USING COVERING INDEX idx_impression_video_user
  [7<-0] SEARCH v USING COVERING INDEX idx_t3_video_owner (owner_id=? AND rowid=?)
  [14<-0] SEARCH vs USING COVERING INDEX idx_t3_seg_impression (impression_id=?)
TIME order C impression,video,segment: rows=1  runs(s)=6.800, 6.835, 6.994  median=6.835s
       result_hash=5f8f164255
```

Answer

- **3.2 Cheapest by formula:** sort-merge at the default memory (Impression is already in key order, so only ViewSegment is sorted); hash join is second; index nested loop is much larger in the worst case. With $M = 100{,}000$ pages (410 MB) the hash join and sort-merge both fall to $b_r + b_s$.
- **Does the engine agree?** No: it uses an index nested loop, because it has no other methods. It is still fast because the lookups walk the Impression tree in order (0 backward steps in 5.7 million rows).
- **J2:** the temporary index is rebuilt in every run; the permanent index removes almost all of the time.
- **3.3 Estimates:** the uniform estimate is about three times too low (owners are not uniform); the creator-aware estimate is 17 to 19 percent low. Both give the same decision.
- **Who is right:** with the three helper indexes the engine chooses my order A and it is about 1,000 times faster than orders B and C. Without indexes the engine is forced into order B, which is slow.

6. Task 4: Predicate Pushdown (15 marks)

Question

Take a query different from A1 C.3 with a selective WHERE applied after a join.

- **4.1** Naive algebra tree, pushed-down tree, the equivalence rule used, and one condition where it does not hold. (7)
- **4.2** Both versions as literal SQL with the run-time difference; if there is none, show from the plan that the optimizer already pushes the filter. (8)

Code

Query: users who were shown the videos of creator 12 (the "shown" part of F6). A1 C.3 rewrote F2, so this query is different. The WHERE keeps 30 of 1,000,000 videos.

```
Naive (join first, filter after; MATERIALIZED stops SQLite from merging the pieces):
  WITH j AS MATERIALIZED (SELECT i.user_id, v.owner_id FROM Impression i
                          JOIN Video v ON v.video_id = i.video_id)
  SELECT DISTINCT user_id FROM j WHERE owner_id = 12;
Pushed down (filter Video first, then join):
  WITH vv AS MATERIALIZED (SELECT video_id FROM Video WHERE owner_id = 12)
  SELECT DISTINCT i.user_id FROM vv JOIN Impression i ON i.video_id = vv.video_id;
As written (the optimizer decides):
  SELECT DISTINCT i.user_id FROM Impression i JOIN Video v ON v.video_id = i.video_id
  WHERE v.owner_id = 12;
```

Command: `python bench.py task4`. Each version is run with no helper index and with `Impression(video_id, user_id)`.

Results

4.1 Equivalence rules and the heuristic.

1. Selection cascade: $\sigma_{c_1 \wedge c_2}(E) = \sigma_{c_1}(\sigma_{c_2}(E))$.
2. Selections commute: $\sigma_{c_1}(\sigma_{c_2}(E)) = \sigma_{c_2}(\sigma_{c_1}(E))$.
3. Selection with product gives a join: $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$.
4. Commuting selection with join: if c uses only attributes of E_1, then $\sigma_c(E_1 \bowtie E_2) = \sigma_c(E_1) \bowtie E_2$.
5. Projection pushdown: $\pi_L(E_1 \bowtie_\theta E_2) = \pi_L(\pi_{L_1}(E_1) \bowtie_\theta \pi_{L_2}(E_2))$ with L_1, L_2 the attributes needed.

Heuristic, step by step: (1) start with one selection over the product; (2) split the conjunction (rule 1); (3) push `owner_id = 12` onto Video (rules 2, 4); (4) put the most restrictive leaf (30 rows) first; (5) product plus join condition become a join (rule 3); (6) push projections (rule 5); (7) pipeline the join.

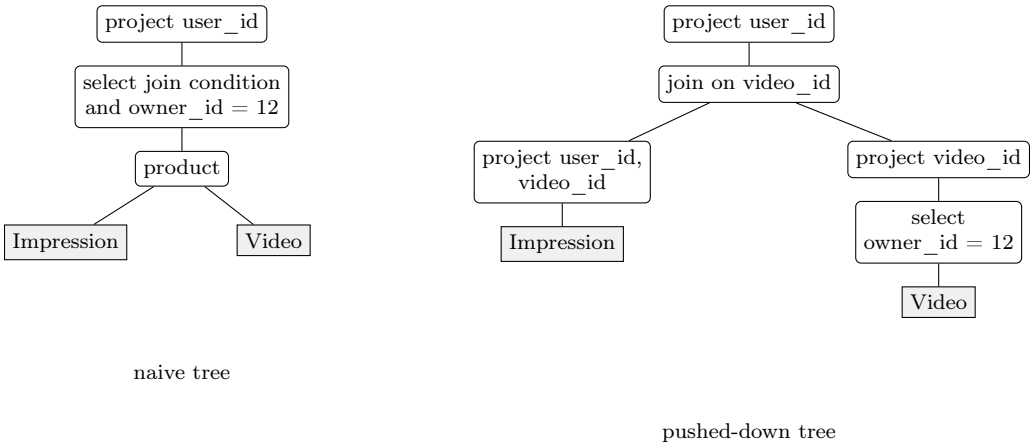


Figure 2. Naive tree (left) and pushed-down tree (right).

Where the rule does not hold: outer join, filter on the null-supplying side.

```
WHERE after LEFT JOIN: 7,092 rows   <- SELECT COUNT(*) FROM Turn t LEFT JOIN UserRating ur ON ur.turn_id
        = t.turn_id WHERE ur.thumbs = 'down'
filter pushed INTO the null-supplying side: 350,275 rows   <- SELECT COUNT(*) FROM Turn t LEFT JOIN
        (SELECT * FROM UserRating WHERE thumbs = 'down') ur ON ur.turn_id = t.turn_id
same WHERE with an INNER join: 7,092 rows   <- SELECT COUNT(*) FROM Turn t JOIN UserRating ur ON
        ur.turn_id = t.turn_id WHERE ur.thumbs = 'down'
```

4.2 Plans and run times.

```
EXPLAIN QUERY PLAN as written (optimizer)
  [4<-0] SCAN i
  [6<-0] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [17<-0] USE TEMP B-TREE FOR DISTINCT

TIME as written (optimizer): rows=451  runs(s)=65.915, 65.669, 65.543  median=65.669s
        result_hash=6e1759760c

EXPLAIN QUERY PLAN pushed
  [3<-0] MATERIALIZE vv
  [6<-3] SCAN Video
  [18<-0] SCAN i
  [24<-0] BLOOM FILTER ON vv (video_id=?)
  [33<-0] SEARCH vv USING AUTOMATIC COVERING INDEX (video_id=?)
  [46<-0] USE TEMP B-TREE FOR DISTINCT

TIME pushed: rows=451  runs(s)=1.084, 1.079, 1.143  median=1.084s  result_hash=6e1759760c

EXPLAIN QUERY PLAN naive
  [3<-0] MATERIALIZE j
  [7<-3] SCAN i
  [9<-3] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [22<-0] SCAN j
  [32<-0] USE TEMP B-TREE FOR DISTINCT

TIME naive: rows=451  runs(s)=69.180, 68.757, 69.174  median=69.174s  result_hash=6e1759760c
```

With the index `Impression(video_id, user_id)`:

```
EXPLAIN QUERY PLAN as written (optimizer)
  [4<-0] SCAN v
  [8<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=?)
  [19<-0] USE TEMP B-TREE FOR DISTINCT

TIME as written (optimizer): rows=451  runs(s)=0.075, 0.080, 0.076  median=0.076s
        result_hash=6e1759760c

EXPLAIN QUERY PLAN pushed
  [3<-0] MATERIALIZE vv
  [6<-3] SCAN Video
  [20<-0] SCAN vv
  [22<-0] SEARCH i USING COVERING INDEX idx_impression_video_user (video_id=?)
  [35<-0] USE TEMP B-TREE FOR DISTINCT
```

```
TIME pushed: rows=451  runs(s)=0.074, 0.074, 0.077  median=0.074s  result_hash=6e1759760c

EXPLAIN QUERY PLAN naive
  [3<-0] MATERIALIZE j
  [7<-3] SCAN i USING COVERING INDEX idx_impression_video_user
  [9<-3] SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
  [22<-0] SCAN j
  [32<-0] USE TEMP B-TREE FOR DISTINCT

TIME naive: rows=451  runs(s)=2.779, 2.730, 2.598  median=2.730s  result_hash=6e1759760c
```

Median time (s)	No helper index	With index
As written (optimizer)	65.67	0.076
Naive (join first)	69.17	2.730
Pushed down (filter first)	1.084	0.074

Answer

- All versions return the same 451 users (same hash).
- **No index:** SQLite does **not** push the filter down. Its plan scans Impression, looks up Video per row, and tests `owner_id` last (same as the naive SQL). The hand-pushed SQL is 60.6 times faster; it scans Impression once against a 30-entry temporary index.
- **With the index:** the optimizer reverses the loops by itself (`SCAN v` first, then `SEARCH i USING COVERING INDEX`). The plan is the evidence that the pushdown happened, and its time equals the pushed SQL.
- The naive SQL stays slow with the index because `MATERIALIZED` forces the whole 15.4 million row join to be built first.
- **Outer join:** pushing the filter into the null-supplying side changes the result (see counts above); a WHERE that rejects NULL lets SQLite turn the join into an inner join first (plan: `SCAN ur`, `SEARCH t`).

7. Task 5: Concurrency: Schedules, Anomalies, Isolation (20 marks)

Question

- **5.1** Precedence graph of the interleaving of A1 transaction T2 (moderation decision); conflict-serializable? equivalent serial order? (5)
- **5.2** A different interleaving of two operations on my own schema that is not conflict-serializable; name the anomaly. (5)
- **5.3** Real demo with two concurrent connections; prevent it with `BEGIN IMMEDIATE`; show that the same interleaving no longer produces it; two sentences on the cost. (10)

Code

`python concurrency_demo.py` (also `python bench.py task5`). Two connections run in two threads on a small database built from `schema.sql` (WAL mode). Python events force the interleaving. The same steps for two interactive shells are in `concurrency_demo.sql`. SQLite in WAL mode: **one writer at a time**; readers use a snapshot and neither block nor are blocked; a deferred `BEGIN` that reads and then writes after another connection committed fails with "database is locked"; `BEGIN IMMEDIATE` takes the write lock first and other writers wait.

Results

5.1 T2 as a schedule. E = table VideoModerationEvent. Schedule: $W_A(E)$ $R_B(E)$ C_A. Only conflict: $R_B(E)$ before $W_A(E)$.

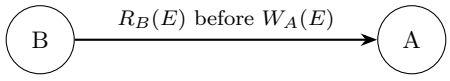


Figure 3. Precedence graph of T2: one edge, no cycle.

```
[    89.5 ms] A  BEGIN
[    89.6 ms] A  INSERT VideoModerationEvent(video 1, 'taken_down')   [write W_A(E)]
[   102.5 ms] B  SELECT current state  [read R_B(E)] -> 'live'   (A has not committed: B still sees the
        old row)
[   102.6 ms] B  INSERT [write W_B(E)] -> error: database is locked   (WAL allows ONE writer at a time)
[   103.4 ms] A  COMMIT
[   108.1 ms] B  SELECT current state  [read R_B(E)] -> 'taken_down'   (after the commit B sees the new
        row)
```

Conclusion 5.1: no cycle, so the schedule is conflict-serializable; the equivalent serial order is B, then A. B reads the last committed snapshot ("live") and is not blocked; its write is refused while A holds the lock.

5.2 New interleaving: handle change limit (A1 T3 rule: at most 2 changes; changes = number of Handle rows of the user − 1). User 20 has made 1 change. Two requests arrive together; H = Handle rows of user 20.

Step	Request A	Request B
1	$R_A(H)$: changes = 1, allowed	
2		$R_B(H)$: changes = 1, allowed
3	$W_A(H)$: end handle, insert handle_from_A	
4	commit	
5		$W_B(H)$: end handle, insert handle_from_B
6		commit

Conflicts: $R_A(H)$ before $W_B(H)$ gives $A \to B$; $R_B(H)$ before $W_A(H)$ gives $B \to A$.

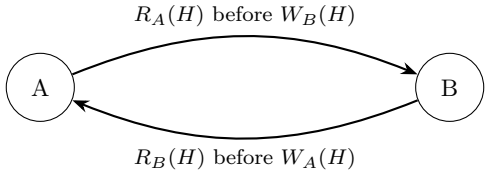


Figure 4. Precedence graph of the handle race: a cycle.

Anomaly: write skew (a check-then-act race). Either serial order refuses the second request, so no serial order gives this result. This is my own scenario on my own tables.

5.3 Demo, fix and cost.

(a) Bad outcome: the check is a plain read outside the transaction.

```
[   185.0 ms] A  check (autocommit read): changes so far = 1, limit 2 -> allowed
[   196.9 ms] B  check (autocommit read): changes so far = 1, limit 2 -> allowed
[   202.4 ms] A  write transaction committed
[   204.0 ms] B  write transaction committed
RESULT 5.3a: handle changes for user 20 = 3 (limit 2) -> LIMIT BROKEN
       ('first', '2026-01-01T00:00:00Z', '2026-03-01T00:00:00Z')
       ('second', '2026-03-01T00:00:00Z', '2026-09-10T10:00:00Z')
       ('handle_from_A', '2026-09-10T10:00:00Z', '2026-09-10T10:00:05Z')
       ('handle_from_B', '2026-09-10T10:00:05Z', None)
```

(b) Check inside a plain `BEGIN`: SQLite refuses the stale write.

```
[   284.3 ms] A  BEGIN (deferred); check inside: changes = 1 -> allowed
[   295.8 ms] B  BEGIN (deferred); check inside: changes = 1 -> allowed
[   303.5 ms] A  write + COMMIT ok
[   303.7 ms] B  write -> database is locked   (B's snapshot is stale; SQLite refuses the write)
[   303.8 ms] B  RETRY: BEGIN, check again: changes = 2 -> refused
RESULT 5.3b: handle changes for user 20 = 2 (limit 2) -> limit kept
       ('first', '2026-01-01T00:00:00Z', '2026-03-01T00:00:00Z')
       ('second', '2026-03-01T00:00:00Z', '2026-09-10T10:00:00Z')
       ('handle_from_A', '2026-09-10T10:00:00Z', None)
```

(c) Fix: `BEGIN IMMEDIATE`, then check, then write. Same interleaving, no bad outcome.

```
[   378.9 ms] B  BEGIN IMMEDIATE ... (blocks, busy timeout 5 s)
[   379.4 ms] A  BEGIN IMMEDIATE (write lock taken); check: changes = 1 -> allowed
[   689.5 ms] A  write + COMMIT ok
[   707.3 ms] B  got the lock after waiting 328 ms; check: changes = 2 -> refused (limit reached)
RESULT 5.3c: handle changes for user 20 = 2 (limit 2) -> limit kept
       ('first', '2026-01-01T00:00:00Z', '2026-03-01T00:00:00Z')
       ('second', '2026-03-01T00:00:00Z', '2026-09-10T10:00:00Z')
       ('handle_from_A', '2026-09-10T10:00:00Z', None)
```

Wait of the second writer for different lock hold times:

```
 lock held (ms)  median wait (ms)  extra over hold (ms)  predicted by backoff (ms)
            1.6              14.5                  12.9                          3
           51.9              57.0                   5.2                         53
          101.5             120.6                  19.0                        103
          301.5             334.5                  33.0                        328
         1001.6            1094.1                  92.6                       1028
The waiter does not wake the moment the lock is free. It sleeps in growing steps (1, 2, 5, 10, 15, 20,
       25, 25, 25, 50, 50, 100 ms ...), so it is late by up to one step.
```

Cost under load (threads add 1 to a counter 100 times each; "lost" = vanished increments, "retries" = refused transactions run again):

```
 threads mode        expected  final  lost  retries  total s  median ms   p95 ms
       2 autocommit       200    106    94        0     0.28       1.12     2.56
       2 deferred         200    200     0      142     0.26       1.14     2.57
       2 immediate        200    200     0        0     0.27       1.13     2.40
       4 autocommit       400    100   300        0     0.57       1.14     2.54
       4 deferred         400    400     0      719     0.51       1.13     2.80
```

```
       4 immediate        400    400     0        0     0.57       1.14     2.67
       8 autocommit       800    100   700        0     1.36       1.14     2.65
       8 deferred         800    800     0     3518     0.99       1.14    53.39
       8 immediate        800    800     0        0     1.43       1.14     2.54

Uncontended cost of BEGIN IMMEDIATE (one connection, 300 transactions of read + write, median per
        transaction):
  deferred   median 1.13 ms per transaction
  immediate  median 1.12 ms per transaction
```

Cost of the fix, in two sentences. With `BEGIN IMMEDIATE` the second writer waits at `BEGIN` for as long as the first holds the lock (328 ms in the demo), and every other writer of the database waits too, because WAL allows one writer at a time, while readers never wait and nothing retries. With a plain `BEGIN` nobody waits for the lock, but the loser is refused and must roll back and run again (the retries column).

5.4 Database-level anomaly (extra): a dirty read – possible only with shared cache and `PRAGMA read_uncommitted = 1`.

```
[ 18584.3 ms] A  BEGIN; INSERT VideoModerationEvent(video 1, 'taken_down')   [write W_A(E), not
        committed]
[ 18585.5 ms] B  default isolation: SELECT state -> error: database table is locked:
        VideoModerationEvent   (shared cache: the table is write-locked, so the reader is blocked)
[ 18585.7 ms] B  PRAGMA read_uncommitted = 1; SELECT state -> 'taken_down'   [read R_B(E) in the middle
        of A's write]
[ 18585.7 ms] A  ROLLBACK   (the reviewer withdraws the decision)
[ 18585.8 ms] B  SELECT state -> 'live'
RESULT 5.4: B saw 'taken_down', a value that was never committed and is not in the database now
        ('live'). That is a dirty read.
Schedule: W_A(E)  R_B(E)  Abort_A. Remove the aborted transaction A and B has read a value that no
        committed transaction ever wrote,
so no serial order of the committed transactions explains what B saw.
[ 18666.2 ms] B  ordinary WAL connection, same moment: SELECT state -> 'live'   (B reads the last
        committed snapshot, no waiting)
```

Answer

- T2 of A1 is conflict-serializable (order B, A). The handle race has a cycle and breaks the limit (3 changes, limit 2) when the check is outside the transaction.
- SQLite transactions are serializable (one writer, snapshot readers). Seen as four separate transactions (read, read, write, write) the handle run has no cycle; the cycle appears only when each read is grouped with its write as one logical operation, which is the application's job.
- Prevention: check and write in the same transaction; with `BEGIN IMMEDIATE` the second request waits and then refuses cleanly. A plain `BEGIN` also prevents it, with an error that must be handled.
- A true database-level anomaly (dirty read) exists in SQLite only in the optional shared-cache mode with `read_uncommitted`; ordinary connections read the last committed state.

8. Summary of Results

Task 1	ToolCall scan: 12,425 pages by hand, 12,300 in a real copy (1.02 percent difference). Plans use full scans, primary key lookups and temporary indexes.
Task 2	F1 2.5x, F4 1.7x, F12 24.6x faster (F12: 220.3 s to 8.970 s). The F4 control index (wrong order) is ignored. Bulk inserts are 6 to 18 times slower with an index.
Task 3	Nested loops only. Sort-merge is cheapest by formula at default memory. J2: 199.3 s without and 2.656 s with the index. Three tables: 28.28 s (engine, no indexes) against 0.004 s (engine, with indexes; my order A).
Task 4	No index: as written 65.67 s, naive 69.17 s, pushed 1.084 s. With the index the optimizer pushes by itself (0.076 s). Outer-join push changes 7,092 rows into 350,275.
Task 5	T2 is conflict-serializable (B, A). Handle race: cycle, limit broken (3 changes) when the check is outside the transaction; `BEGIN IMMEDIATE` prevents it; second writer waited 328 ms.

Confirmation run against the previous run

Every experiment was run twice, in two separate sessions on AC power (previous run, and this confirmation run). Plans, row counts and result hashes were identical; times agree within the usual variation.

Median time (s)	Previous run	This run
F1 before / after	0.295 / 0.111	0.289 / 0.115
F4 before / after	0.051 / 0.029	0.051 / 0.030
F12 before / after	222.1 / 8.593	220.3 / 8.970
J2 without / with index	204.1 / 2.679	199.3 / 2.656
Three tables, no index / order A	26.42 / 0.002	28.28 / 0.004
Task 4 as written / pushed (no index)	68.82 / 1.136	65.67 / 1.084

Limits (what is not verified)

- All timings use a warm cache; cold-disk reads were not measured.
- Sort-merge and hash join costs are formulas, because SQLite cannot run them.
- Part of the F12 time after the indexes is not explained.
- Write costs were measured on scratch copies of the first 300,000 rows, with default synchronisation.
- The five rebuilt patch scripts are not the original A1 code.
- The WAL locking and busy-handler description is from the SQLite documentation (from memory); the demos support it. No `sqlite3` shell exists on my machine, so `concurrency_demo.sql` was not typed into two shells; the Python demo was run.

9. Checklist of the Handout Items

Item	Required	Page
Start 1	SCALE = 50, one transaction, foreign keys on, row counts about 50 times A1	1
Start 2	`EXPLAIN QUERY PLAN` and wall-clock timing	1
Start 3	Changelog of A1 changes	1
Start 4	One PDF; repository with `indexes.sql`, `plan_analysis.md`, `concurrency_demo.sql`, updated `queries.sql`, `transactions.sql`	this report; files listed below
Start 5	Reproducible numbers; LLM-use appendix	14
1.1	Plans of three queries; access path of every table	2
1.2	Full-scan arithmetic by hand	3
1.3	One sentence per query	4
2.1	Three indexes, column order by leftmost prefix	4
2.2	Plan and time before / after; diagnosis of ignored index	4
2.3	One rejected index	6
3.1	Two joins and their strategy	7
3.2	Nested-loop, sort-merge, hash cost by formula	7
3.3	Three-table join order	8
4.1	Naive and pushed trees, rule, outer-join failure	9
4.2	Both versions as SQL with run times	10
5.1	Precedence graph of T2	11
5.2	New non-serializable interleaving, anomaly named	11
5.3	Real demo, `BEGIN IMMEDIATE` fix, two sentences on cost	12, cost sentences 13

Repository files: `indexes.sql`, `plan_analysis.md`, `concurrency_demo.sql`, `queries.sql`, `transactions.sql`, `schema.sql`, `generate_data.py`, `concurrency_demo.py`, `bench.py`, `verify_db.py`, `a1_scripts/`, `README.md`. Database files are not committed.

“‘latex

Appendix: Use of an AI Assistant

An AI assistant (Claude Code) was used as a supplementary tool while completing Assignments A1 and A2. My work focused on understanding the assignment requirements, analysing SQL queries, investigating database performance, evaluating indexing strategies, testing alternative approaches, and interpreting the experimental results. AI assistance was used for debugging, exploring implementation alternatives, automating repetitive tasks, and improving report formatting.

For A2, I worked on the database experiments at SCALE = 50, query optimisation, index selection, and performance evaluation. I examined query execution plans using `EXPLAIN QUERY PLAN`, compared execution times under different indexing strategies, and investigated the reasons behind observed performance differences. The benchmarking and concurrency scripts were used to evaluate query performance and demonstrate concurrent database operations. I also reviewed the generated outputs and incorporated the relevant findings into the report.

Testing and Verification: During the assignment, several initial suggestions and results required further investigation:

1. The initial prediction about the best column order for the F1 index was not supported by the measured results. The final choice was based on the observed execution plan and benchmark.
2. The initial explanation of the F12 execution time did not fully account for the observed cost. Additional testing showed that the temporary index explained only part of the execution time.
3. Initial index-size measurements were affected by the reuse of freed database pages. The results were reviewed and corrected.
4. Script execution errors and LaTeX formatting issues were identified through testing, inspection of the generated outputs, and verification of the compiled PDF.
5. Initial index choices for Task 1 were reviewed against the assignment requirements for high-volume tables, and alternative indexes were evaluated.
6. Benchmark timings obtained under inconsistent power conditions were discarded, and the measurements were repeated under consistent conditions.
7. SQL scripts, query plans, and generated outputs were checked for execution errors, consistency, and compliance with the assignment requirements.

Final Review and Responsibility: I reviewed the experimental evidence, investigated discrepancies between initial predictions and actual results, and used the verified outputs to support the conclusions. AI-generated suggestions and code were treated as assistance rather than unquestioned results. The final report was checked against the assignment requirements, and the submitted PDF was compiled using `pdflatex`.

The benchmarking and concurrency scripts and the recorded experimental outputs provide a basis for reproducing and explaining the reported performance comparisons.